Deep Reinforcement Learning for Electric Vehicle Routing Optimization

Energy-Minimizing Electric Vehicle Routing Problem (EM-EVRP)

Dhia Elhak Ezzeddini Salem Fradi

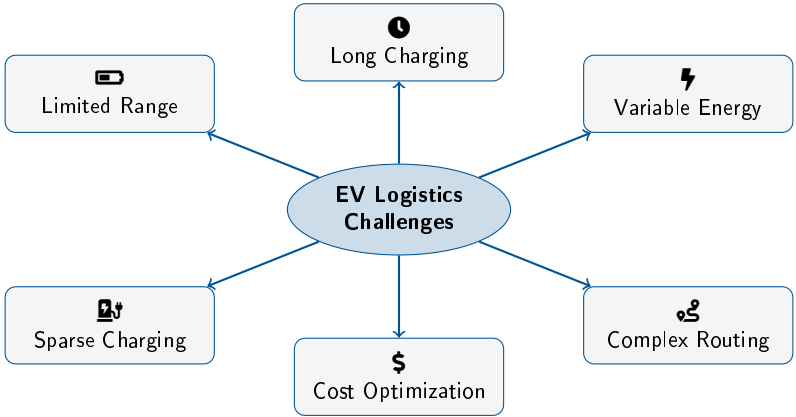
Higher School of Communication of Tunis
Carthage University

Supervisor: Dr. Ferdaoues Chaabane
Host Company: Sagemcom

Academic Year 2025–2026

Outline

Challenges in EV Logistics



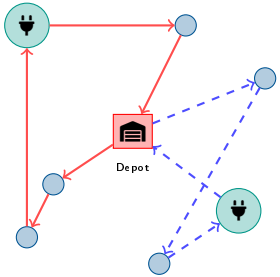
Electric Vehicle Routing Problem (EVRP)

Classical VRP:

- Serve all customers from depot
- Minimize total travel distance/cost
- Respect vehicle capacity

EVRP Extension:

- **Battery capacity** constraints
- **Charging station** visits allowed
- **Energy consumption** varies with load
- **Recharging time** impacts schedule



Multiple routes with charging stops

Energy-Minimizing EVRP (EM-EVRP)

Problem Definition

Objective: Minimize **total energy consumption** while serving all customers

Subject to Constraints:

1. **Customer Service**
   - Each customer visited exactly once
   - Full demand satisfaction
2. **Vehicle Capacity**
   - Total load $\leq$ max capacity
   - Partial deliveries possible
3. **Battery SOC**
   - SOC ≥ 0 at all times
   - Recharge at stations
4. **Time Windows**
   - Service within $[a_i, b_i]$
   - Route time budget
5. **Route Feasibility**
   - Start and end at depot
   - Valid path existence
6. **Charging Logic**
   - Optional station visits
   - Charging time cost

MILP Formulation: Sets and Parameters

Sets

N	All nodes (depot + customers)
C	Customers $C = N \setminus \{0\}$
S	Charging stations
K	Fleet of vehicles
R	Trip indices

Network Parameters

d_i	Demand at customer i
Q_k	Capacity of vehicle k
t_{ij}	Travel time on arc (i, j)
$[a_i, b_i]$	Time window at node i

EV Energy Parameters

m_c	Vehicle curb mass
C_d	Aerodynamic drag coeff.
C_r	Rolling resistance coeff.
ρ	Air density
A	Frontal area
α_{ij}	Slope angle on arc
$\varphi_{d/r}$	Motor efficiency
$\phi_{d/r}$	Battery efficiency

Physics-Based Energy Consumption Model

Total Vehicle Mass on Arc (i, j)

$$m_{ij} = m_c + u_{ij}$$

where u_{ij} is the current cargo load

Mechanical Power Required

$$P_{ij} = \underbrace{\frac{1}{2} C_d \rho A v_{ij}^2}_{\text{Aerodynamic drag}} + \underbrace{m_{ij} g \sin \alpha_{ij}}_{\text{Gravitational}} + \underbrace{m_{ij} g C_r \cos \alpha_{ij}}_{\text{Rolling resistance}}$$

Energy Consumption with Efficiency

$$E_{ij} = \begin{cases} \phi_d \cdot \varphi_d \cdot P_{ij} \cdot t_{ij} & \text{if } P_{ij} \geq 0 \text{ (propulsion)} \\ \phi_r \cdot \varphi_r \cdot P_{ij} \cdot t_{ij} & \text{if } P_{ij} < 0 \text{ (regeneration)} \end{cases}$$

MILP Formulation: Objective and Constraints

Objective: Minimize Total Energy

$$\min \sum_{k \in K} \sum_{r \in R} \sum_{(i,j) \in A} E_{ij} \cdot x_{ijk}^{r}$$

Customer Service: $\sum_{k,r} y_{ik}^{r} = 1$
Capacity: $\sum_{i} d_i y_{ik}^{r} \leq Q_k$

Flow: $\sum_{j} x_{ijk}^{r} = y_{ik}^{r}$
MTZ: $w_{ik}^{r} - w_{jk}^{r} + |C| x_{ijk}^{r} \leq |C| - 1$

NP-Hard $\Rightarrow$ Exact methods don't scale

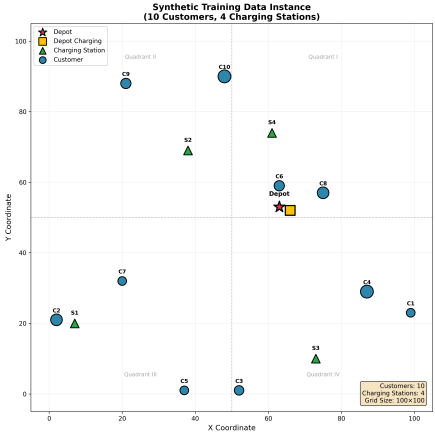
Data Generation Pipeline

Training Data:

- Random node coordinates on 100×100 grid
- Random elevations for slope calculation
- Random customer demands
- Pairwise distances and slopes computed

Dataset Size

Training	1,280,000
Validation	10,000
Batch size	1,024



Classical Optimization Approaches

We implemented **four baseline methods** for comparison:

1. Google OR-Tools

- Industry-standard solver
- Constraint programming
- Arc cost callbacks for energy
- Local search improvements

2. Hill Climbing

- Simple local search
- 2-opt, Or-opt, Exchange
- Fast but local optima

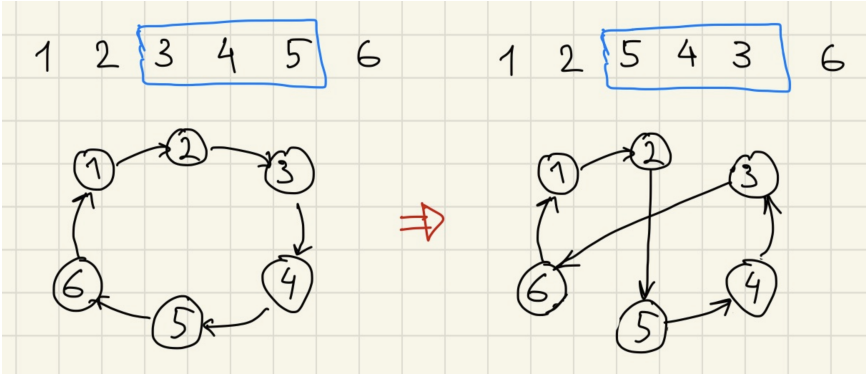
3. Simulated Annealing

- Probabilistic acceptance
- Escapes local optima
- Temperature-based control
- $P(\text{accept}) = e^{-\Delta E/T}$

4. Ant Colony Optimization

- Nature-inspired
- Pheromone trails
- Collective intelligence

Local Search: 2-Opt Operator



2-Opt Improvement

Reverse a segment of the route to eliminate crossing edges and reduce total cost

Simulated Annealing: Acceptance Rule

Probabilistic Acceptance

$$P(\text{accept worse solution}) = \exp\left(-\frac{\Delta E}{T}\right)$$

Temperature Behavior:

- High $T \Rightarrow$ explore (accept worse)
- Low $T \Rightarrow$ exploit (only improvements)

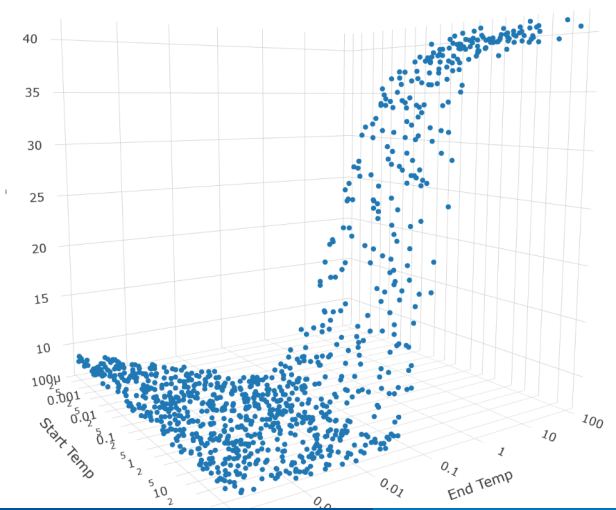
Key Parameters:

- T_{start}: initial temperature
- T_{end}: final temperature
- Cooling: $T_{k+1} = \alpha \cdot T_k$

Challenge

How to choose T_{start} and T_{end} for optimal performance?

SA Parameter Tuning: Initial Exploration



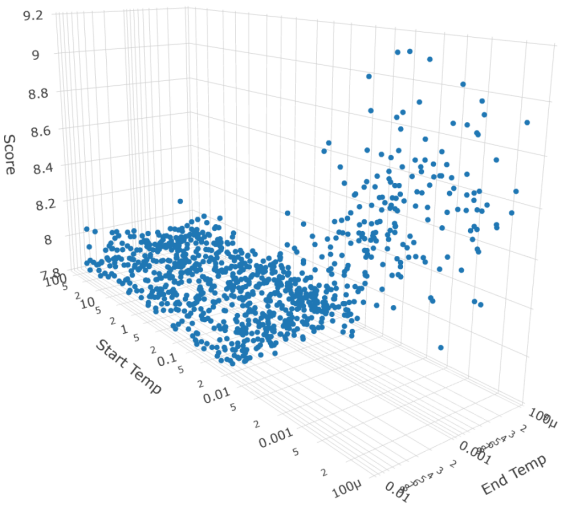
Experiment Setup:

- Monte Carlo sampling
- Vary T_{start} and T_{end}
- Record score distribution

Observation

High variance across parameter space — need to narrow down!

SA Parameter Tuning: Restricting T_{end}



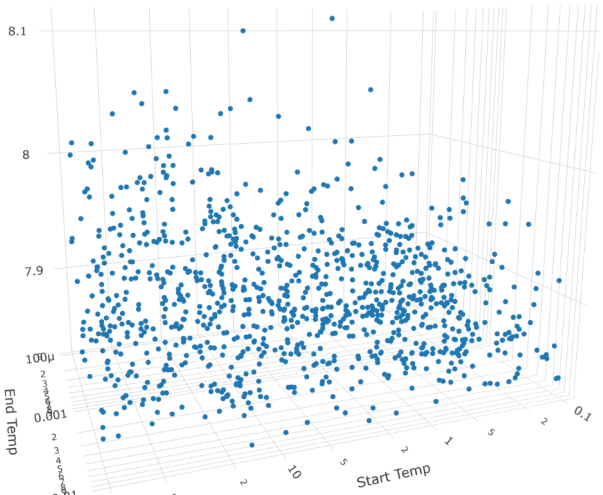
Restriction Applied:

$$T_{\text{end}} < 0.01$$

Finding

Lower end temperatures lead to better convergence — algorithm should cool down sufficiently

SA Parameter Tuning: Restricting T_{start}



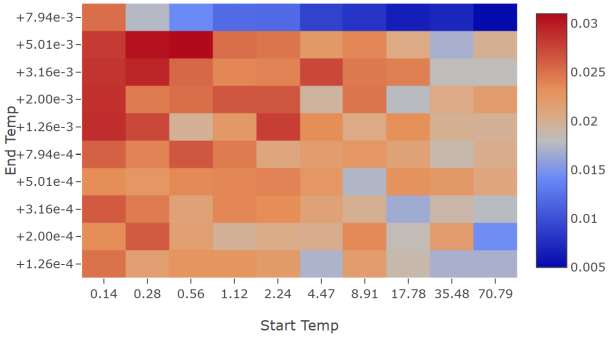
Restrictions Applied:

$$T_{\text{start}} \geq 0.1 \quad \text{and} \quad T_{\text{end}} < 0.01$$

Finding

Starting temperature must be high enough for initial exploration

SA Parameter Tuning: Heatmap Analysis



Methodology:

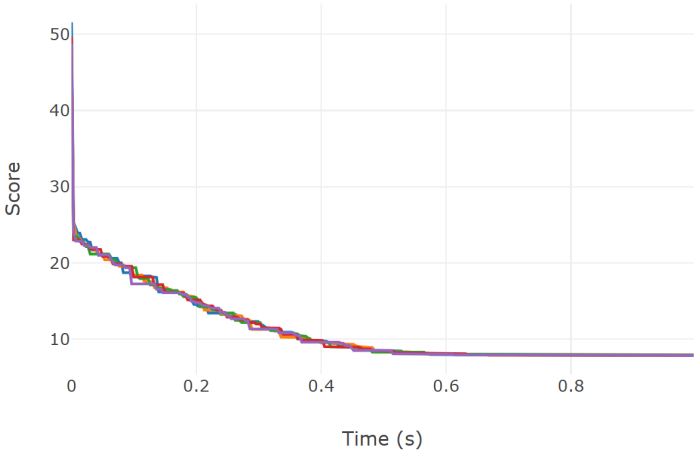
- 10×10 parameter grid
- 5000 Monte Carlo iterations
- Track best solution frequency

Optimal Region

$$T_{\text{start}} \approx 0.2$$

$$T_{\text{end}} \approx 5 \times 10^{-3}$$

SA Convergence with Optimal Parameters



Configuration

$T_{\text{start}} = 0.2, \quad T_{\text{end}} = 5 \times 10^{-3}$

Ant Colony Optimization

Probabilistic Transition Rule:

$$p_{ij}^{a} = \frac{[\tau_{ij}]^{\alpha} \cdot [\eta_{ij}]^{\beta}}{\sum_{l \in \mathcal{N}_i} [\tau_{il}]^{\alpha} \cdot [\eta_{il}]^{\beta}}$$

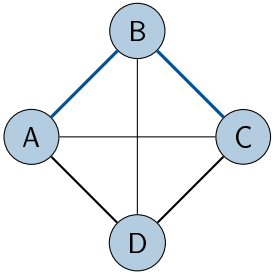
where:

- τ_{ij} = pheromone level on arc (i, j)
- $\eta_{ij} = 1/c_{ij}$ = heuristic information
- α, β = importance parameters

Pheromone Update:

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \sum_{a} \Delta\tau_{ij}^{a}$$

Better solutions deposit more pheromone



Line thickness = pheromone level

Key Properties

- Positive feedback
- Parallel exploration
- Emergent optimization

Why Deep Reinforcement Learning?

Traditional Methods Limitations

- Exact methods: exponential time
- Heuristics: problem-specific design
- No generalization to new instances
- Slow for real-time applications

DRL Advantages

- **Learn** to solve, not program
- **Generalize** to new instances
- **Fast inference** (milliseconds)
- **End-to-end** optimization

Key Insight

Train a neural network to **construct** solutions step-by-step,
learning from experience which decisions lead to lower energy consumption

Inspired by: Kool et al., "Attention, Learn to Solve Routing Problems!" (ICLR 2019)

MDP Formulation for EVRP

Markov Decision Process Components

State s_t: Current partial solution + vehicle status

- Visited nodes, current position
- Remaining capacity, SOC, time budget

Action a_t: Select next node to visit

- Customer, charging station, or depot

Reward r_t: Negative energy for traversed arc

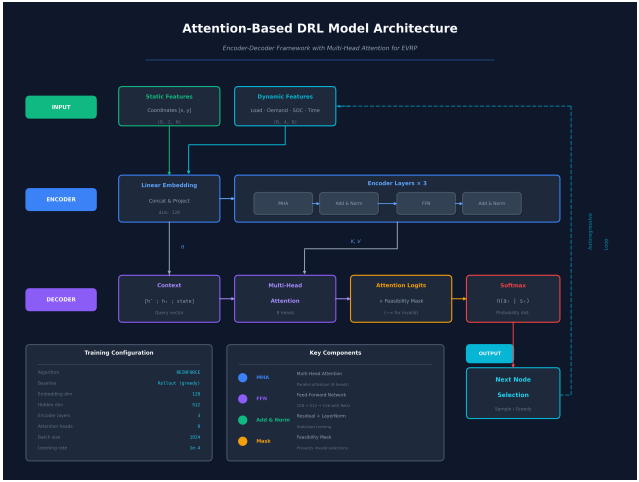
- Minimize cumulative energy consumption

Policy π_θ: Neural network with parameters θ

- Maps states to action probabilities

Objective: $\max_\theta \mathbb{E}_{\pi_\theta}\left[\sum_t r_t\right] = \max_\theta \mathbb{E}_{\pi_\theta}\left[-\sum_t E_t\right]$

Attention-Based Architecture: Overview



Graph Attention Encoder

Multi-Head Self-Attention Layer

$$h_i^{(l)} = \text{BN}\left(h_i^{(l-1)} + \text{MHA}\left(h_i^{(l-1)}, H^{(l-1)}\right)\right)$$

Attention Mechanism:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Graph Embedding:

$$\bar{h} = \frac{1}{n}\sum_{i=1}^{n} h_i$$

Mean aggregation of all node embeddings

Architecture Details

Embedding dim	128
Attention heads	8
Encoder layers	3
FF hidden	512
Normalization	Batch

Context-Aware Decoder

Context Embedding

$$c_t = W_c \left[\bar{h}; h_{\pi_{t-1}}; \text{load}_t; \text{SOC}_t; \text{time}_t\right]$$

Combines: graph embedding + previous node + vehicle state

Attention-Based Selection

$$u_{tj} = \begin{cases} C \cdot \tanh\left(\frac{q_t \cdot k_j}{\sqrt{d_k}}\right) & \text{if } j \in \mathcal{A}_t \\ -\infty & \text{otherwise} \end{cases}$$

$$p_\theta(a_t = j | s_t) = \text{softmax}(u_t)_j$$

Feasibility Masking

$\mathcal{A}_t$ = nodes that satisfy: demand $\leq$ capacity AND energy needed $\leq$ SOC

REINFORCE Training Algorithm

Policy Gradient Objective

$$J(\theta) = \mathbb{E}_{\pi_\theta}[L(\pi)] \quad \text{where } L(\pi) \text{ is route energy}$$

Gradient with Rollout Baseline

$$\nabla_\theta J \approx \frac{1}{B} \sum_{i=1}^{B} (L(\pi_i) - b(s_i)) \nabla_\theta \log p_\theta(\pi_i | s_i)$$

$b(s)$ = baseline from greedy policy copy (reduces variance)

Baseline Update

Update when policy shows significant improvement (paired t-test, $p < 0.05$)

EV-Specific Model Features

State of Charge Tracking

$$\text{SOC}_{t+1} = \text{SOC}_t - \frac{E_{ij}}{E_{\text{battery}}}$$

- Continuous SOC monitoring
- Reset at charging stations
- Negative consumption = regeneration

Load Dynamics

$$\text{load}_{t+1} = \text{load}_t - d_{\text{served}}$$

Cargo decreases after each delivery

Dynamic Masking Rules

1. Sufficient SOC for arc?
2. Can reach depot from there?
3. Enough capacity for demand?
4. Time budget remaining?
5. Station detour feasible?

Learning Charging Decisions

Model learns **when** to visit charging stations through experience

Real-World Deployment: OSRM Integration

Open Source Routing Machine (OSRM):

- Production-grade routing engine
- Real street-level distances
- Fast table API for distance matrices
- OpenStreetMap data

Integration Pipeline:

1. Upload customer GPS coordinates
2. Query OSRM for distance matrix
3. Run DRL model inference
4. Reconstruct routes on map
5. Calculate real energy consumption

Scalability Strategy

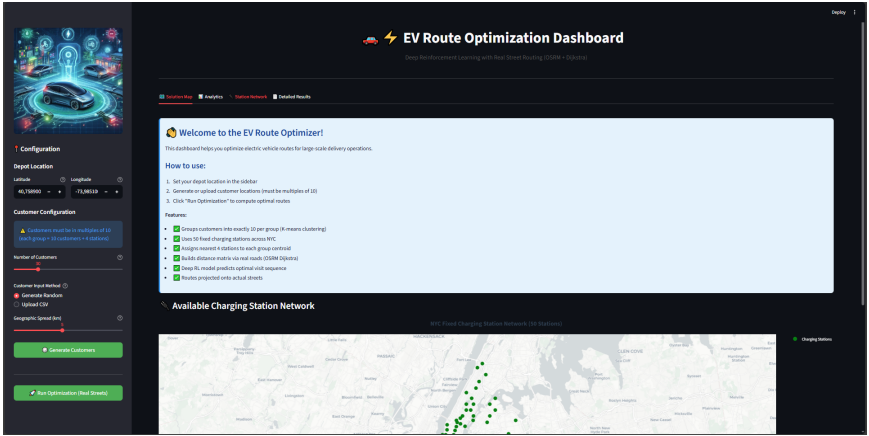
K-means Clustering:

- Group customers into batches
- 10 customers per group
- Run DRL on each group
- Combine results

NYC Configuration

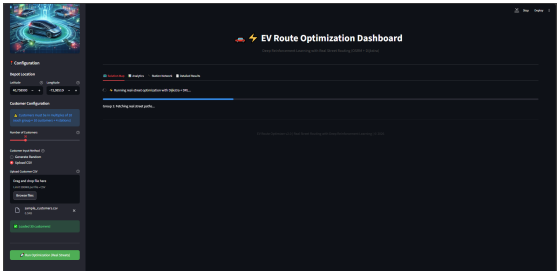
50 pre-configured charging stations across New York City

Interactive Dashboard: Main Interface

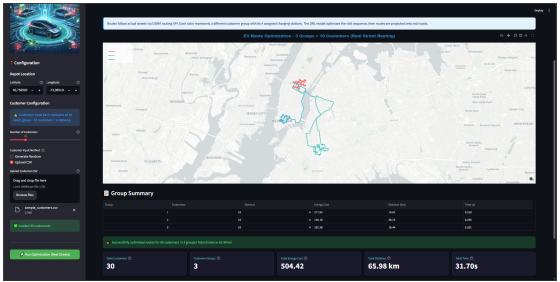


Upload customers • Configure EV • Select stations • Real-time optimization

Interactive Dashboard: Route Optimization



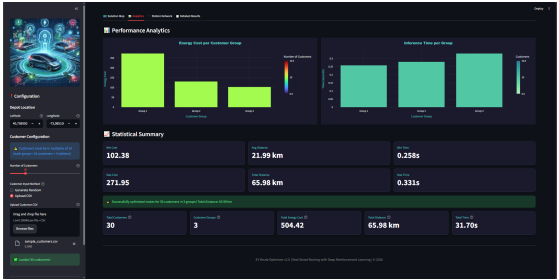
Inference Process

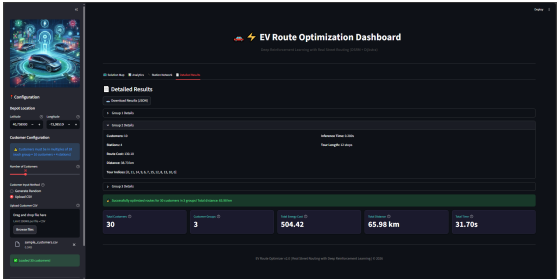


Optimized Routes

- Interactive map with customer locations and charging stations
- Real-time route visualization with street-level paths
- Color-coded routes for multiple vehicles

Interactive Dashboard: Analytics





Analytics Features

- Energy consumption breakdown per route segment
- SOC tracking throughout the journey
- Cost analysis and performance metrics
- Export results as JSON for integration

Training Configuration

Problem Configuration

Customers	10
Charging stations	4
Sequence length	16
Initial SOC	80%
Max load	50 units

Training Parameters

Epochs	20
Batch size	1,024
Training samples	1,280,000
Learning rate	10^{-4}
Baseline	Rollout

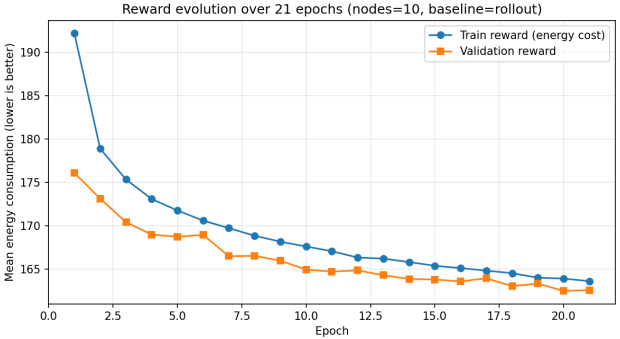
Model Architecture

Embedding dim	128
Encoder layers	3
Attention heads	8
FF hidden	512
Tanh clipping	10
Optimizer	Adam

Hardware

GPU	NVIDIA RTX
Training time	$\sim$8 hours

Training Results



Observations

Consistent improvement, convergence at epoch 80

Generalization

Validation follows training, no overfitting

Performance Comparison

Method	Energy (kWh)	Inference Time
OR-Tools	**11.92**	2.1s
DRL (Ours)	12.18	**0.5s**
Ant Colony Optimization	12.67	3.2s
Simulated Annealing	12.89	1.5s
Hill Climbing	13.21	0.8s

DRL Performance

- Only **2.2%** gap vs OR-Tools
- **7.8%** better than Hill Climbing
- Near-optimal solution quality

Speed Advantage

- **4x** faster than OR-Tools
- **6x** faster than ACO
- Real-time capable

Key Findings

Model Performance

- ✓ Generalizes to unseen instances
- ✓ Competitive solution quality
- ✓ Significantly faster inference
- ✓ Learns charging decisions

Energy Model Insights

- Load-dependent consumption
- Regenerative braking benefits
- Elevation-aware routing

Architecture Effectiveness

- Attention captures relationships
- Context embedding is crucial
- Masking ensures feasibility

Practical Benefits

- Ready for deployment
- Interactive visualization
- Real street routing
- Scalable to larger problems

Summary of Contributions

1. **Comprehensive MILP Formulation**
   - Physics-based energy consumption model
   - Multi-trip capability with time windows
   - Regenerative braking consideration
2. **Baseline Implementation**
   - OR-Tools, Hill Climbing, SA, ACO
   - Standardized benchmarking framework
3. **Deep Reinforcement Learning Solution**
   - Attention-based encoder-decoder
   - EV-specific state representation
   - Fast and generalizable
4. **Production-Ready System**
   - Interactive Streamlit dashboard
   - OSRM integration for real streets
   - Export and analytics features

Limitations and Future Work

Current Limitations

- Trained on 10 customers
- Single depot scenario
- Homogeneous fleet
- Static demand
- No traffic dynamics

Future Directions

- Scale to 50+ customers
- Multi-depot extension
- Heterogeneous EV types
- Dynamic reoptimization
- Real-time traffic integration
- Joint routing + charging scheduling

Research Extensions

- Transfer learning for larger instances
- Multi-agent coordination for fleet management
- Integration with smart grid for optimal charging times

Thank You!

Backup: References (1/2)

- W. Kool, H. van Hoof, M. Welling, *"Attention, Learn to Solve Routing Problems!"*, ICLR, 2019.
- M. Tang et al., *"Energy-optimal routing for EVs using DRL with transformer,"* Applied Energy, 2023.
- T. Erdelić, T. Carić, *"A Survey on the EVRP: Variants and Solution Approaches,"* J. Adv. Transportation, 2019.
- S. Kirkpatrick et al., *"Optimization by Simulated Annealing,"* Science, 1983.
- M. Dorigo et al., *"The Ant System: Optimization by cooperating agents,"* IEEE Trans. SMC, 1996.
- G. B. Dantzig, J. H. Ramser, *"The Truck Dispatching Problem,"* Management Science, 1959.

Backup: References (2/2)

- E. W. Dijkstra, *"A note on two problems in connexion with graphs,"* Numerische Mathematik, 1959.
- J. MacQueen, *"Some methods for classification and analysis of multivariate observations,"* Berkeley Symp., 1967.
- A. Vaswani et al., *"Attention Is All You Need,"* NeurIPS, 2017.
- Google OR-Tools. `https://developers.google.com/optimization`
- Project OSRM. `http://project-osrm.org/`
- CVRPLIB. `http://vrp.atd-lab.inf.puc-rio.br/`
- G. Boeing, *"OSMnx: Street Networks,"* Computers, Environment and Urban Systems, 2017.